NumPy Coding Practice

7 Hands-On Problems for Data Science

Covering: Arrays, ndarray, Data Types, Broadcasting, Vectorization & Random Functions

Problem 1: Array Boot Camp

Master the fundamentals of creating and inspecting NumPy arrays. This problem covers array creation from lists, built-in generators, and exploring array metadata.

Tasks

1. Create a 1D array from the Python list [10, 20, 30, 40, 50] and print its shape, dtype, and ndim.
2. Create a 2D array of shape (3, 4) filled with zeros, then another filled with ones.
3. Use np.arange() to create an array from 5 to 50 with a step of 5.
4. Use np.linspace() to generate 20 evenly spaced values between 0 and 1.
5. Create a 4x4 identity matrix using np.eye().

Starter Code

Problem 2: Data Type Detective

NumPy supports many data types. In this problem you will create arrays with specific dtypes, convert between them, and observe how dtype affects memory usage.

Tasks

1. Create an integer array [1, 2, 3] with dtype=np.int8, then print its itemsize.
2. Create the same array with dtype=np.float64 and compare the itemsize.
3. Create a complex number array: [1+2j, 3+4j, 5+6j].
4. Convert the float64 array to int32 using .astype() and verify the change.
5. Create a boolean array by checking which elements of np.array([15, 22, 8, 33, 5]) are greater than 10.

Starter Code

Problem 3: Reshape & Slice Master

Working with array shapes and slicing is essential for data manipulation. Practice reshaping, transposing, and extracting subsets of data.

Tasks

1. Create an array of numbers 1 to 12, then reshape it into a (3, 4) matrix.
2. Extract the second row of the matrix.
3. Extract the third column of the matrix.
4. Extract the 2x2 sub-matrix from the bottom-right corner.
5. Transpose the matrix and verify the new shape is (4, 3).

Starter Code

Hint: Remember: Python indexing starts at 0. Use -2: for last two rows/columns.

Problem 4: Broadcasting Challenge

Broadcasting allows NumPy to perform operations on arrays of different shapes without explicit loops. This is one of NumPy's most powerful features.

Tasks

1. Create a (3, 3) matrix of ones and add the 1D array [10, 20, 30] to each row.
2. Create a column vector [[1], [2], [3]] and multiply it with a row vector [10, 20, 30] to get a (3, 3) result.
3. Normalize each column of a (4, 3) random matrix by subtracting the column mean and dividing by the column standard deviation.
4. Add a scalar value 100 to every element of a (2, 5) matrix (scalar broadcasting).

Starter Code

Problem 5: Vectorization vs. Loops

Replace slow Python loops with fast vectorized NumPy operations. This problem demonstrates why vectorization matters for performance in Data Science.

Tasks

1. Create an array of 1 million random numbers. Square every element WITHOUT using a loop.
2. Given two arrays a and b of the same length, compute their element-wise sum, product, and the Euclidean distance between them - all without loops.
3. Use np.where() to replace all negative values in an array with 0 (vectorized conditional).
4. Use time measurement to compare the speed: a Python loop vs. vectorized squaring of 1M elements.

Starter Code

Problem 6: Random Number Generator Toolkit

Master NumPy's random module - the essential toolkit for simulations, sampling, and machine learning. Practice with rand, randn, randint, uniform, and normal distributions.

Tasks

1. Use np.random.rand() to create a (3, 4) array of uniform random numbers in [0, 1).
2. Use np.random.randn() to create a (5, 5) array of standard normal (mean=0, std=1) random numbers.
3. Use np.random.randint() to generate 10 random integers between 1 and 100 (inclusive).
4. Use np.random.uniform(low=5.0, high=15.0, size=1000) and compute the mean (should be close to 10).
5. Use np.random.normal(loc=70, scale=10, size=500) to simulate exam scores, then compute mean, std, min, and max.

Starter Code

Problem 7: Mini Data Analysis Project

Combine everything you learned! Simulate a dataset of student exam scores, perform a full statistical analysis, and apply data transformations using NumPy only.

Tasks

1. Generate a (100, 3) matrix representing 100 students' scores in 3 subjects: Math (mean=75, std=12), Science (mean=68, std=15), English (mean=80, std=8).
2. Clip all scores to the range [0, 100] using np.clip().
3. Compute the per-student average (mean across columns), then find the top 5 students by average score.
4. Find the subject with the highest overall mean and the one with the most variance.
5. Normalize all scores to a 0-1 range using min-max normalization: (x - min) / (max - min) per column.

Starter Code

Good luck and happy coding!

Tip: Run each problem in a Jupyter Notebook cell for instant feedback.